Rod Cutting Problem

COMPLETE TECHNICAL DOCUMENTATION & ANALYSIS

The Rod Cutting Problem is one of the classic Dynamic Programming problems and is actually an application of the Unbounded Knapsack Problem.

Problem Statement

Suppose we have a rod of length N.

For every possible length, we know the price obtained by selling that piece.

Example:

Length	Price
1	2
2	5
3	7
4	8
5	10

Rod length = 5

We can cut the rod into pieces in any way. Our goal is to **Find the maximum revenue (price) obtainable**.

Example Scenarios for Rod Length = 5:

- **Cut: 5** → Revenue: 10
- **Cut: 1 + 4** → Revenue: 2 + 8 = 10
- **Cut: 2 + 3** → Revenue: 5 + 7 = 12
- **Cut: 1 + 1 + 3** → Revenue: 2 + 2 + 7 = 11
- **Cut: 2 + 2 + 1** → Revenue: 5 + 5 + 2 = 12

Maximum revenue = 12

Why is this Unbounded Knapsack?

Recall the structure of the Unbounded Knapsack problem compared directly to the Rod Cutting problem:

Knapsack Feature	Rod Cutting Equivalent
Items	Lengths
Weight[]	Length[]
Value[]	Price[]
Capacity W	Rod length N
Can reuse item unlimited times	Can cut same length unlimited times

Example Mapping

Suppose:

- Length = [1, 2, 3, 4, 5]
- Price = [2, 5, 7, 8, 10]
- Rod length = 5

This strictly maps to:

- Weight[] = [1, 2, 3, 4, 5]
- Value[] = [2, 5, 7, 8, 10]
- Capacity = 5

This is exactly the same as the Unbounded Knapsack problem structure.

Important Observation

The index itself represents a rod length. Suppose:

```
price = [2, 5, 7, 8, 10]
```

Then:

- index 0 → length 1
- index 1 → length 2
- index 2 → length 3
- index 3 → length 4
- index 4 → length 5

Therefore, we can formalize the relation as:

$$rodLength = index + 1$$

Constraint

We cannot exceed the total rod length. Suppose remaining rod length:

```
N = 5
```

Current index:

```
i = 2
```

Length represented:

```
rodLength = i + 1 = 3
```

Can we take it? Since $3 \leq 5$, **Yes**.

After taking:

```
Remaining length = 5 - 3 = 2
```

Why does index not decrease after taking?

Because the same cut length can be used again. Suppose Length 2 gives price 5. We may choose a breakdown like 2 + 2 + 1. Therefore, after taking:

```
take = price[i] + solve(i, N - (i + 1))
```

Notice that we call $solve(i, ...)$ and not $solve(i - 1, ...)$. This is exactly why it is called an Unbounded Knapsack problem.

Recursion Idea

At every index, we have exactly two choices:

1. **Skip current length:** Move to smaller lengths.

   ```
   skip = solve(i - 1, N)
   ```

2. **Take current length:** Allowed only if $(i + 1) \leq N$. Then:

   ```
   take = price[i] + solve(i, N - (i + 1))
   ```

Finally, we take the maximum of both paths:

```
return max(take, skip);
```

Recursive Tree Example

Suppose:

```
price = [2, 5, 7, 8]
Rod length = 4
```

Current state: *solve(3, 4)* (index 3 → length 4)

- **Skip 4:** Moves evaluation to *solve(2, 4)*
- **Take 4:** Revenue: *8 + solve(3, 0)*. Since rod length becomes zero, it returns **8**.

Now explore *solve(2, 4)* (index 2 → length 3):

- **Skip:** *solve(1, 4)*
- **Take:** *7 + solve(2, 1)*

Eventually, we obtain all combinations:

- 4 → 8
- 3 + 1 → 7 + 2 = 9
- 2 + 2 → 5 + 5 = 10
- 2 + 1 + 1 → 5 + 2 + 2 = 9
- 1 + 1 + 1 + 1 → 2 + 2 + 2 + 2 = 8

Maximum value obtained is **10**. Thus, answer = 10.

Building the Base Case

This is where Rod Cutting differs slightly from Unbounded Knapsack.

Unbounded Knapsack Base Case:

```
if(i == 0)
{
    return (W / wt[0]) * val[0];
}
```

Because item 0 can be taken many times to fully fill weight capacity.

Rod Cutting Base Case:

At index 0, the length represented is 1 and the price is $price[0]$.

Suppose remaining rod length = N. Since length 1 can be used infinitely ($1 + 1 + 1 + 1 + ...$):

- Number of cuts = N
- Revenue = $N \times price[0]$

Hence, the base case is defined as:

```
if(i == 0)
{
    return N * price[0];
}
```

Example:

Suppose $price[0] = 2$ and remaining rod length = 4. The only possible cut layout is $1 + 1 + 1 + 1$. Revenue: $2 + 2 + 2 + 2 = 8$. Therefore: $4 \times 2 = 8$.

Complete Recursive Code

```
int solve(int i, int N, vector<int>& price)
{
    // base case
    if(i == 0)
    {
        return N * price[0];
    }

    // skip current length
    int skip = solve(i - 1, N, price);

    int take = INT_MIN;
    int rodLength = i + 1;

    // take current length
    if(rodLength <= N)
    {
        take = price[i] + solve(i, N - rodLength, price);
    }

    return max(take, skip);
}
```

Initial Call instruction: $solve(n - 1, n, price);$

Memoization Version

```
int solve(int i, int N, vector<int>& price, vector<vector<int>>& dp)
{
    if(i == 0)
    {
        return N * price[0];
    }

    if(dp[i][N] != -1)
        return dp[i][N];

    int skip = solve(i - 1, N, price, dp);

    int take = INT_MIN;
    int rodLength = i + 1;

    if(rodLength <= N)
    {
        take = price[i] + solve(i, N - rodLength, price, dp);
    }

    return dp[i][N] = max(take, skip);
}
```

State Definition

> $dp[i][N]$ **means:** Maximum revenue obtainable using lengths from 1 to $(i + 1)$ to fill rod length N.

Example: $dp[2][5]$ means using lengths $\{1, 2, 3\}$ to construct rod length 5, determining the absolute maximum price.

Similarity Matrix with Unbounded Knapsack

Unbounded Knapsack	Rod Cutting
Weight array	Length array
Value array	Price array
Capacity W	Rod length N
Take → same index	Take → same index
Skip → i - 1	Skip → i - 1
Base = (W / wt[0]) * val[0]	Base = N * price[0]
Maximize value	Maximize price

Pattern

Whenever you observe the following properties:

- Unlimited reuse allowed ✓
- Need maximum profit/value ✓
- Capacity restriction ✓

Think: **Unbounded Knapsack Pattern**. Rod Cutting is simply an Unbounded Knapsack scenario where the weights are rod lengths and the values are prices.

Final Intuition

Think of a rod as a knapsack:

- Rod length = Capacity
- Cut length = Weight
- Price of cut = Value

Every time you make a cut, you are essentially "putting an item into the knapsack". Because the same length can be used repeatedly, the problem statement cleanly translates to:

"Find the combination of lengths whose total length does not exceed N and whose total price is maximum."

That is exactly the Unbounded Knapsack Problem.